Zero-Knowledge Zero Trust

Nikita Borisov and Yupeng Zhang
{nikita,zhangyp}@illinois.edu

October 21, 2024

PIs

Nikita Borisov, Professor, Electrical and Computer Engineering
University of Illinois at Urbana-Champaign
Tel: 217-244-5385, Email: nikita@illinois.edu

Yupeng Zhang, Assistant Professor, Electrical and Computer Engineering
University of Illinois at Urbana-Champaign
Tel: 217-244-7595, Email: zhangyp@illinois.edu

Abstract

This project proposes a novel approach to implementing Zero Trust architecture in Operational Technology (OT) environments, addressing the unique challenges posed by devices with limited connectivity and computational capabilities. We introduce two key innovations: (1) using verifiable computing to support flexible authorization policies in low-power devices, and (2) a programmatic approach to authorization checks using universal circuits. We will also address the challenges of authentication and auditing by making use of low-trust authentication adapters and log transparency.

Neeeds Assessment Area: A. Zero Trust Networking

Project Description

1 Introduction

Zero Trust architecture is a security model that assumes that all users, devices, and applications are potentially hostile and therefore should be treated as such. It therefore applies authentication, authorization, and auditing to all interactions. In a typical enterprise, this is typically supported by a centralized identity provider, using protocols such as OAuth, OpenID Connect, and SAML. The provider can perform complex authentication checks, such as using multi-factor authentication, use complex authoriation policies, such as role-based access control, and use machine learning to detect anomalous behavior in the system audit. Additionally, using a centralized provider makes it easier to deploy new types of policies, technologies, and mechanisms

This security model is difficult to adapt to the OT environment, where devices do not have the ability to connect to a centralized identity provider. We therefore propose to make use of zero-knowledge proofs and verifiable computing to enable a zero-trust security model in the OT environment. In particular, we will make use of low-trust *authentication adapters* to enable sophisticated new authentication mechanisms to be used in the OT environment, and *log transparency* to ensure that auditing is properly performed. The adapters and logs will produce *succinct* proofs, which can be efficiently verified by devices with low computational capabilities.

For the purposes of authorization, we will make use of *universal circuits* to enable a programmatic approach to authorization checks. Universal circuits enable a prover to attest to the correct execution of a certified program without the verifier needing to understand the internals of the program. This means that a centralized authority can create a complex authorization policy, such as role-based access control or attribute-based access control, and create a certified implementation of the policy that can be used in the OT environment. The device can then verify the correctness of the policy as it is executed, without having to understand the policy itself. In this way, entirely new *types* of policies can be deployed without the need to upgrade the OT device, which is often infeasible due to the constraints of the OT environment.

2 Background: Zero Knowledge Proofs and SNARKs

Zero-knowledge proofs are a cryptographic primitive that allows one party (the prover) to prove to another party (the verifier) that a certain statement is true, without revealing any information except for the fact that the statement is indeed true. A typical format for the statement is that a common input (known to both prover and verifier) x satisfies a predicate $P(x)$. The predicate is typically something that is difficult for the verifier to compute directly, such as an NP-hard problem. In seminal work, Goldreich, Micali, and Wigderson showed that zero-knowledge proofs are possible for any NP language [17]. However, their construction was not practical, and only more recently have zero-knowledge proofs become efficient enough to be used in applications.

An important development has been the construction of *succinct* zero-knowledge proofs [15, 18, 19], also called *succint non-interactive arguments of knowledge or SNARKs*, which are proofs that are both zero-knowledge and have small proof sizes. SNARKs can be used to efficiently verify that a remote computation was performed correctly while using only a small amount of communication and computational resources, typically independent of the size of the computation. In SNARKs the computation is typically represented as a kind of circuit, but SNARKs can also be constructed for C-like programs [2].

SNARKs require a trusted setup step, where a set of parameters are generated that are used to create the proof. This setup was originally dependent on the predicate $P(x)$, but recent advances in the field have designed *universal* SNARKs, which allow the verifiable execution of arbitrary circuits or programs [7, 8, 26, 28]. These operate by either encoding a universal circuit in the trusted setup parameters, or by using having the prover demonstrate the correct execution of a virtual machine (zkVM) on a program.

3 Authorization Approaches

Authorization policies describe what users, devices, or services are allowed to do. Legacy OT systems often use simple access control lists. For example, a SCADA system may have an `operator` user who is allowed to access a control panel, a `monitor` user who is allowed to view the current state of the system, and an `admin` user who is allowed to modify configuration. Each of these users may be authenticated using a separate password.

This approach has many disadvantages, from a lack of flexibility to password management and reuse headaches. We can contrast this with a modern Zero Trust architecture, which uses an *identity provider (IdP)* to manage user accounts and roles, and a *policy engine (PE)* to manage access control policies. In such an approach, a user first authenticates to the identity provider to procure an access token. Then, a request to access a system passes through a *policy enforcement point (PEP)*, where the policy engine uses the role information associated with the access token to make an authorization decision. See Figure 1 for a diagram of this architecture.

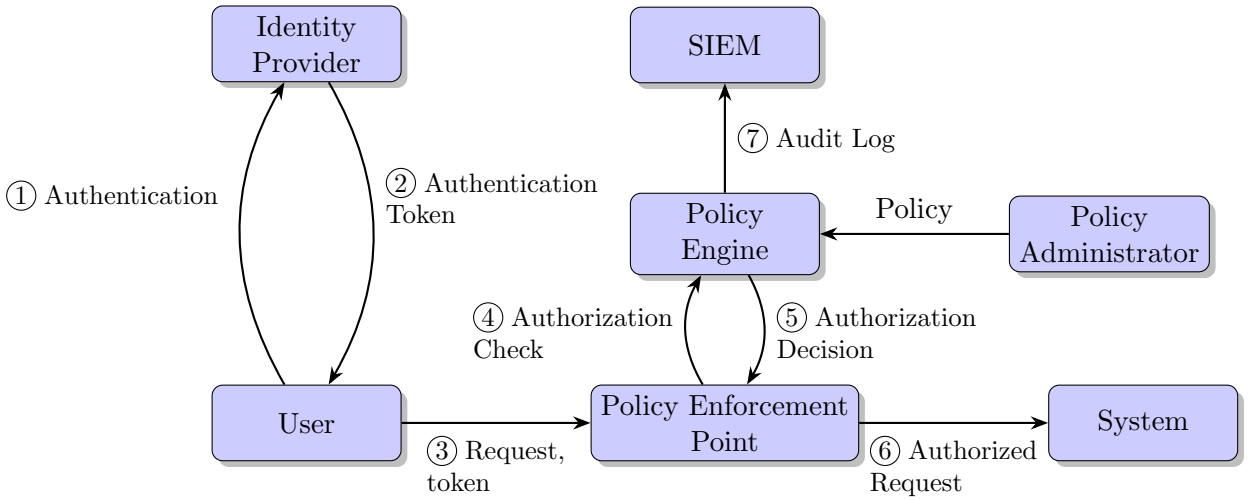

Figure 1: Zero Trust architecture

The identity provider can implement sophisticated authentication mechanisms, such as multi-factor authentication, and the policy engine can implement complex access control policies, such as role-based access control or attribute-based access control, that utilize both user attributes and request context. The policy engine can also provide an audit log of all requests to a *security information and event management (SIEM)* system. By separating these functions into distinct components, it is easier to incorporate new technologies and mechanisms as they are developed.

This centralized approach, however, is difficult to implement in an OT environment as it requires complex online interactions between the user, the identity provider, and the policy engine. An alternative approach is to use a *decentralized* authorization architecture, where the authorization

policy is encoded into one or more *certificates* that are distributed to users and devices [1, 13, 22]. While traditionally certificates are used to associate public keys with identities, they can also include policy statements, such as "user alice is authorized for the admin role" and "users with operator roles can access the control panel". These statements are signed by a policy authority, which is the equivalent of a policy administrator. Complex decentralized authorization logics [3, 4, 12, 24] can combine a variety of statements and support delegation of authority.

In the past, decentralized authorization have seen very limited adoption, due to a number of challenges:

1. **Certificate management**: Certificates need to be distributed to users and devices, and kept up to date as policies change.
2. **Policy management**: As policies are distributed among a variety of certificates, it can be difficult to understand the resulting composite policy, effect changes, and analyze its security properties.
3. **Policy privacy**: Since policies are encoded in certificates, they are visible to any party that can access them. This can leak sensitive information and allow adversaries to identify weak points in the policy.
4. **Key management**: Most decentralized systems require each user and device to hold one or more private keys, which must be securely stored and protected from compromise. Key updates and revocation can be difficult to manage.
5. **Implementation complexity**: Complex decentralized systems require potentially expensive operations to be performed on end systems. Additionally, proper implementation of these systems is challenging and errors can introduce serious security vulnerabilities. For example, the handling of X.509 certificates in embedded systems has been called "the most dangerous code in the world" [16] due to the prevalence of exploitable parsing and logic errors.
6. **Lack of auditability**: Decentralized systems make it difficult to audit the resulting policy and accesses, as the audit log is distributed among potentially many devices.
7. **Understanding context**: Fine-grained authorization policies must incorporate the context of the request to distinguish different types of operations (e.g., checking SCADA system status versus changing the system configuration) that should be authorized differently. This application-specific and often complex context needs to be somehow incorporated into the policy logic, making it difficult to use generic off-the-shelf solutions.

In previous work, PI Borisov proposed a *programmable* authorization architecture, which addresses the last challenge by encoding the policy logic as a function, executed by a Java Virtual Machine (JVM) [6]. The function takes as input a request, as signed by the user's private key, and makes an authorization decision based on the request contents and user identity. In essence, active certificates create an executable version of policy statements used in decentralized logics, which allows for maximally expressive policy logic. Certificates can also be composed together to support delegation of authority and modular policies.

The use of off-the-shelf JVMs can help reduce the implementation complexity, and advances such as JIT compilation can help reduce the runtime overhead. Nevertheless, JVMs are inherently complex and will require security updates and patches to address new vulnerabilities. Resource constraints in the OT environment are also a concern, as programs in malicious certificates may intentionally degrade system performance, and even benign ones may incur a significant performance overhead.

4 Zero-knowledge Authorization

The use of zero-knowledge proofs and verifiable computing can help address many of the challenges associated with decentralized authorization. In particular, a user can prove to a device that they are authorized to access a resource in a succinct and easy-to-verify manner. To start this, we can think of the following authorization language:

$$L = \{(R, u, P) \mid \mathsf{Evaluate}(P, R, u) = 1\}$$

where R is the request, u is the user's identity, P is the policy, and $\mathsf{Evaluate}(P, R, u)$ is a function that evaluates the policy P given the request R and user identity u. We extend this to include an authentication of the user's identity and the policy:

$$L = \{(R, u, x_U, P, \sigma_P) \mid \mathsf{Auth}(u, x_U) = 1 \wedge \mathsf{Verify}(P, \sigma_P, PK_{PA}) = 1 \wedge \mathsf{Evaluate}(P, R, u) = 1\}$$

where x_U is the user's authentication secret (e.g., private key) that can be matched to its (public) identity u using the Auth function, and σ_P is a signature on the policy P by the policy authority PA with public key PK_{PA}, as verified by the Verify function.

To carry out a request, a user needs to generate a proof that they are authorized to access the resource. In particular, they need to produce a proof of knowledge of (u, x_U, P, σ_P) such that $(L, u, x_U, P, \sigma_P) \in L$. Note that proof of knowledge of an authenticator is the classic identification problem [14,25], and verifying signatures in zero-knowledge is well-studied in the context of anonymous credentials [5,9,10]. The key, then, is to design a zero-knowledge proof system for the policy evaluation predicate $\mathsf{Evaluate}(P, R, u)$.

Our initial approach will be to implement a ZK proof for an expressive policy language, such as industry standard XACML [11] or an academic language [20,23]. We will make use of an existing SNARK tools, such as `libsnark` or `zoKrates`, to generate the proofs and evaluate the feasibility of this approach. Note that one of the main challenges will be to design an encoding of the policy language that is amenable to efficient zero-knowledge proof generation.

Our next step will be to implement $\mathsf{Evaluate}$ using a zkVM. The universality will allow the policy to be fully general, both in terms of the policy language and interpreting the request context. The use of a zkVM will allow for efficient execution of the policy, as the circuit only needs to be compiled once and all execution will be performed in the zkVM.

Both these designs will allow the policy to be highly expressive while introducing minimal complexity on the device, which need only verify succinct proofs. The use of generic zkVMs also allows for the deployment of entirely new policy languages and abstractions without requiring any changes to the device.

5 Adapters

The zero-knowledge approach reduces the complexity of supporting expressive and evolving policies by the device. However, it requires that the user generate proofs, which can be computationally expensive. Part of our work will be to optimize the efficiency of such proof generation, but we expect the costs to remain significant. Additionally, many of the challenges of decentralized authorization enumerated above remain.

To address these challenges, we will make use of *adapters*, which will be responsible for generating proofs and performing other tasks such as authenticating the user's identity, managing policies, and collecting audit logs. At a first glance, the Adapter may seem like it replicates the

functionality of the identity provider, policy engine, and SIEM. However, key to our approach is to use zero-knowledge proofs to make the adapter *low trust*. The adapter need not be correct, as its functionality can be verified by the device; this enables adapters to be more easily replicated and deployed in a variety of locations, including behind the DMZ.

The use of signed policies, together with the proofs in the previous section, means that the adapter need not be trusted to correctly implement policy evaluation—an error in the adapter can only result in an invalid proof, which the device will reject. We will also investigate the use of zero-knowledge proofs to verify the identity management and audit functions of the adapter.

For identity management, public key certificates can be used to associate a user identity with one or more private keys stored on their devices. We will also look into creating a verifiable execution of password and multi-factor authentication; for example, we can use the `sphinx` [27] protocol to allow adapters to verify passwords without risk of their disclosure, and use zero-knowledge proofs to verify the correctness of the password checks. Likewise, one-time-use authentication tokens can also be cryptographically verified by the adapter.

To support auditing, the adapter can log all requests to a tamper-evident log, such as those used in certificate transparency [21]. The logs can provide proofs (or promises) of logging that can then be verified by the device before executing the request.

6 Project Milestones and Future Work

The expected duration of the project is one year, with the following quarterly milestones:

1. **Q1** Identify a policy language and sample policies that would be appropriate for the OT environment.

2. **Q2** Implement a zero-knowledge proof for the policy evaluation predicate using an existing SNARK tool.

3. **Q3** Optimize the proof generation from Q2 and start work on zkVM implementation. Develop an initial proof of log inclusion for audit records.

4. **Q4** Implement a proof-of-concept zkVM policy evaluator and evaluate the performance overhead of both approaches. Integrate the log inclusion proofs from Q3 with the ZK policy proofs.

We leave the implementation of low-trust identity management for future work. Another important area of future research is optimizing the zkVM approach for the authentication scenarios. In particular, we believe there is significant potential for performance improvements by designing a specialized zkVM that can operate directly on policies and request context.

References

[1] Moritz Y. Becker and Peter Sewell. Cassandra: Distributed access control policies with tunable expressiveness. In *Proceedings of the 5th IEEE International Workshop on Policies for Distributed Systems and Networks (POLICY 2004)*, pages 159–168, 2004.

[2] Eli Ben-Sasson, Alessandro Chiesa, Daniel Genkin, Eran Tromer, and Madars Virza. SNARKs for C: Verifying program executions succinctly and in zero knowledge. In *Annual Cryptology Conference*, pages 90–108. Springer, 2013.

[3] Matt Blaze, Joan Feigenbaum, and Angelos D. Keromytis. The KeyNote trust-management system version 2. RFC 2704, September 1999. `https://www.ietf.org/rfc/rfc2704.txt`.

[4] Matt Blaze, Joan Feigenbaum, and Jack Lacy. Decentralized trust management. In *Proceedings of the 1996 IEEE Symposium on Security and Privacy*, pages 164–173. IEEE, 1996.

[5] Dan Boneh and Xavier Boyen. Short signatures without random oracles. In *Advances in Cryptology—EUROCRYPT 2004*, pages 56–73. Springer, 2004.

[6] Nikita Borisov and Eric A. Brewer. Active certificates: A framework for delegation. In Paul C. van Oorschot and Virgil Gligor, editors, *9th Network and Distributed System Security Symposium (NDSS)*, pages 155–165, Reston, VA, USA, February 2002. Internet Society.

[7] Sean Bowe, Ariel Gabizon, and Ian Miers. Scalable zero knowledge via cycles of elliptic curves. In *Annual International Cryptology Conference*, pages 276–305. Springer, 2017.

[8] Sean Bowe, Jack Grigg, and Daira Hopwood. Halo infinite: Recursive zk-snarks from any homomorphic encryption. *Cryptology ePrint Archive*, 2020:1536, 2020.

[9] Jan Camenisch and Anna Lysyanskaya. An efficient system for non-transferable anonymous credentials with optional anonymity revocation. In *Advances in Cryptology—EUROCRYPT 2001*, pages 93–118. Springer, 2001.

[10] Jan Camenisch and Markus Stadler. Proof systems for general statements about discrete logarithms. Technical Report 260, Department of Computer Science, ETH Zurich, 1997.

[11] Erik Rissanen (Editor). extensible access control markup language (XACML) version 3.0. OASIS Standard, January 2013.

[12] Carl M. Ellison, Brian Frantz, Butler W. Lampson, Ron Rivest, Brian M. Thomas, and Tatu Ylönen. SPKI certificate theory. RFC 2693, September 1999. `https://www.ietf.org/rfc/rfc2693.txt`.

[13] Stephen Farrell and Ramaswamy Chandramouli. An introduction to attribute certificates. *IEEE Security & Privacy*, 1(4):45–49, 2003.

[14] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Advances in Cryptology—CRYPTO'86*, pages 186–194. Springer, 1987.

[15] Rosario Gennaro, Craig Gentry, and Bryan Parno. Quadratic span programs and succinct nizks without pcps. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 626–645. Springer, 2013.

[16] Martin Georgiev, Subodh Iyengar, Suman Jana, Rishita Anubhai, Dan Boneh, and Vitaly Shmatikov. The most dangerous code in the world: validating ssl certificates in non-browser software. In *Proceedings of the 2012 ACM conference on Computer and communications security*, pages 38–49, 2012.

[17] Oded Goldreich, Silvio Micali, and Avi Wigderson. Proofs that yield nothing but their validity or all languages in np have zero-knowledge proof systems. *Journal of the ACM (JACM)*, 38(3):690–728, 1991.

[18] Jens Groth. Short non-interactive zero-knowledge proofs. In Antoine Joux, editor, *Advances in Cryptology – EUROCRYPT 2009*, volume 5479 of *Lecture Notes in Computer Science*, pages 341–358. Springer, 2009.

[19] Jens Groth. On the size of pairing-based non-interactive arguments. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 305–326. Springer, 2016.

[20] Xin Jin, Ram Krishnan, and Ravi Sandhu. A unified attribute-based access control model covering DAC, MAC, and RBAC. In *Proceedings of the 17th ACM Symposium on Access Control Models and Technologies (SACMAT)*, pages 65–74, 2012.

[21] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency. In *2013 IEEE Symposium on Security and Privacy*, pages 317–327. IEEE, 2013.

[22] Ninghui Li, John C. Mitchell, and William H. Winsborough. Design of a role-based trust-management framework. In *Proceedings of the 2002 IEEE Symposium on Security and Privacy*, pages 114–130, 2002.

[23] Jaehong Park and Ravi Sandhu. The UCON abc usage control model. *ACM Transactions on Information and System Security*, 7(1):128–174, 2004.

[24] Ronald L. Rivest and Butler W. Lampson. SDSI – a simple distributed security infrastructure. Technical Report MIT/LCS/TR-692, Massachusetts Institute of Technology, 1996. `http://people.csail.mit.edu/rivest/pubs/RL96.pdf`.

[25] Claus-Peter Schnorr. Efficient identification and signatures for smart cards. In *Advances in Cryptology—CRYPTO'89*, pages 239–252. Springer, 1990.

[26] Srinath Setty. Nova: Recursive zero-knowledge arguments from folding schemes. In *Annual International Cryptology Conference*, pages 359–388. Springer, 2022.

[27] Maliheh Shirvanian, Stanislaw Jareckiy, Hugo Krawczykz, and Nitesh Saxena. Sphinx: A password store that perfectly hides passwords from itself. In *2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS)*, pages 1094–1104. IEEE, 2017.

[28] Alin Tomescu, Michael Straka, Riad S. Wahby, Christopher W. Fletcher, and Srinivas Devadas. zkVM: A zero-knowledge virtual machine for private smart contracts. In *2020 IEEE Symposium on Security and Privacy Workshops (SPW)*, pages 106–112. IEEE, 2020.

PROJECT BUDGET CITES I/UCRC

P.I. / P.D.: Nikita Borisov
Co-Investigator: Yupeng Zhang
AGENCY/SPONSOR:
TITLE: Zero-knowledge Zero Trust
BUDGET PERIOD: Jan 1, 2025–Dec 31, 2025

INDIRECT COST RATES:	**11.11% max rate for projects**
INDIRECT COST:	11.11%
TUITION REMISSION:	64.00%

CATEGORY		# of RAs	SALARY RATE	% / HRS	# mos	Period I	Total
SALARIES and WAGES:							
P.I.:	Nikita Borisov		$21,111	4%	12	$10,133	$10,133
	Yupeng Zhang		$15,777	4%	12	$7,573	$7,573
				100%		$0	$0
				100%		$0	$0
	RSCH ASSTS	1.00	$2,980	100%	11	$32,780	32,780
	SUBTOTAL:					$50,486	$50,486
FRINGE BENEFITS:				RATES			
P.I.:	Nikita Borisov			42.32%		$4,288	$4,288
	Yupeng Zhang			42.32%		$3,205	$3,205
				42.32%		$0	$0
				42.32%		$0	$0
	RSCH ASSTS:			9.82%		$3,219	3,219
	SUBTOTAL:					$10,712	$10,712
	TOTAL SALARIES AND FRINGE:					$61,198	$61,198
TRAVEL:	**Domestic**					2,000	2,000
OTHER DIRECT COSTS:							
	MATERIALS AND SUPPLIES:					0	0
	PUBLICATIONS:					0	0
	COMPUTER SERVICES:					0	0
	GENERAL SERVICES: communications					0	0
	TUITION REMISSION:					20,979	20,979
	TOTAL OTHER DIRECT COST:					$20,979	$20,979
TOTAL DIRECT COSTS:						$84,177	$84,177
INDIRECT COST BASE:						*$63,198*	*$63,198*
TOTAL INDIRECT COST:						$7,021	$7,021
TOTAL COST OF PROJECT:						**$91,198**	**$91,198**